# Lean Agent Code & Multi-Agent Systems

Ponytail · gitea-mcp · Claude Code · Self-Hosted LLMs

MQS · mark@mqs.dk

2026

Modular Quantum Solutions

# Lean Agent Code

# Why Code Quality Matters for AI Agents

**The problem**: AI agents tend to over-engineer — adding layers, abstractions, and boilerplate that create maintenance debt without adding value.

**Research findings on AGENTS.md / code-guidance files**:

| Intervention | Compliance |
|---|---|
| Documentation alone | ~25–40% |
| Runtime enforcement hooks | ~95% |
| Auto-generated AGENTS.md | *reduces* success rate by ~3% |

**Conclusion**

Rules without enforcement drift. The solution is a decision ladder baked into every coding session.

# Ponytail Decision Ladder

Before writing a single line of code, an AI agent asks:

1. **Does it need to exist?** — skip if not required
2. **Already in codebase?** — reuse existing code
3. **Stdlib handles it?** — prefer standard library
4. **Native platform feature?** — use platform primitives
5. **Installed dependency?** — use what's already there
6. **One line?** — write one line
7. **Then**: minimum viable implementation

**Non-negotiable**

Validation, error handling, security, and accessibility are always kept — ponytail targets complexity, not safety.

# Ponytail in Practice — QPUBench Examples

Fixes made during QPUBench development under ponytail review:

| File | Issue | Fix |
|---|---|---|
| `store.py` `ParquetStore.query` | `pd.Series` mask — bug + unnecessary | Replaced with `df = df[df[col] == val]` loop |
| `stub.py` `StubMBQCAdapter.spec` | `pattern = None` — dead variable | Removed |
| `adapter.py` (qforte) | `tuple([sym, tuple(xyz)])` — list wrapped in tuple | Changed to `(sym, tuple(xyz))` |
| `store.py _iter` | No return type | Added `-> Iterator[BenchmarkRecord]` |
| `record.py _utcnow` | Lambda assigned to name (noqa E731) | Converted to typed `def` |

# Ponytail — Install & Use

```
# Claude Code plugin
/plugin install ponytail@ponytail

# Review current diff for over-engineering
/ponytail-review

# Scan entire repo for unnecessary code
/ponytail-audit

# Collect deferred optimisation shortcuts
/ponytail-debt
```

**Supported agents**: Claude Code · OpenAI Codex · GitHub Copilot CLI · Cursor · Windsurf · Gemini CLI · Devin

# Multi-Agent System

# Why Multi-Agent?

**Single-agent limitation**: one agent, one context window, one repo at a time.

**Multi-agent enables**:

- Parallel work across multiple repos or services
- Specialised agents per domain (schemas · adapters · tests · docs)
- Continuous background tasks (CI watching, PR review, dependency updates)
- Self-hosted pipelines with no cloud data exposure

```
User
 └── Orchestrator agent
        ├── Schema agent     (qpubench schemas)
        ├── Adapter agent    (integrations/)
        └── Review agent     (tests + AGENTS.md checks)
```

# gitea-mcp Integration

**gitea-mcp** connects AI agents directly to Gitea repositories via MCP

```
AI Agent (Claude Code / Cursor / Windsurf)
    └── MCP protocol
            └── gitea-mcp server  (gitea.com/gitea/gitea-mcp)
                    └── Gitea REST API
                            └── repos · issues · PRs · wikis · CI
```

- Official MCP server from the Gitea project
- Agents can read/write code, open PRs, comment on issues, trigger CI
- Works with Claude Code, Mistral Vibe, Gemini CLI, OpenCode, Devin
- Self-hosted Gitea = full data sovereignty

# Supported Agents & Platforms

| Agent | Open-weight | Self-hostable | gitea-mcp |
|---|---|---|---|
| Claude Code (Anthropic) | No | No | Yes |
| Mistral Vibe | Yes | Yes | Yes |
| Gemini CLI (Google) | No | No | Yes |
| GitHub Copilot CLI | No | No | Yes |
| OpenCode | Depends | Yes | Yes |
| Devin | No | No | Yes |

All agents read the same `AGENTS.md` file — one set of rules, any tool.

# Claude Code + Mistral Vibe

**Claude Code** *(Anthropic)*

- CLI-first agentic coding assistant
- `CLAUDE.md` / `AGENTS.md` for persistent rules
- Git worktrees for isolated parallel agents
- Per-project memory system across sessions
- Hooks for automated validation

**Mistral Vibe** *(Mistral AI)*

- Open-weight models, self-hostable
- MCP-compatible via gitea-mcp
- Suitable for offline / air-gapped environments
- No data leaves your infrastructure

Both integrate with the same `AGENTS.md` instruction file.

# Context & Memory Management

**Per-session context discipline**:

| File | Purpose |
|---|---|
| `AGENTS.md` | Language-agnostic, permanent agent instructions |
| `CLAUDE.md` | Claude-specific rules, project conventions |
| `CONTEXT.md` | Current task, status, constraints — written before each session |

**Workflow**:

- Before starting → write `CONTEXT.md` with current task + constraints
- Each session → `"Read CONTEXT.md and CLAUDE.md first"`
- When switching repos → update `CONTEXT.md` with what was just completed
- Parallel agents → clear boundary definitions + shared `contract.ts`

# Multi-Agent Memory Architecture

**Four persistent memory types** (survive across sessions):

| Type | Content | When to save |
|---|---|---|
| `user` | Role, expertise, preferences | First conversation |
| `feedback` | Corrections + validated approaches | After any feedback |
| `project` | Goals, deadlines, decisions | When context changes |
| `reference` | External system pointers | When a resource is named |

```
~/.claude/projects/<repo>/memory/
├── MEMORY.md            <- index (always loaded, max 200 lines)
├── user_role.md
├── feedback_testing.md
└── project_schema.md
```

# Self-Deployed LLMs

**Abzu** (HuggingFace) — open-weight models for quantum computing tasks

- Local deployment via HuggingFace `transformers` / Ollama / vLLM
- No data leaves your infrastructure
- Suitable for air-gapped laboratory environments
- Combine with self-hosted Gitea + gitea-mcp for a fully sovereign pipeline

```
Local LLM (Ollama / vLLM)
    └── MCP client (Claude Code / Cursor)
            └── gitea-mcp  -->  Gitea (self-hosted)
                                    └── qpubench repo
```

**Full self-hosted stack**

Model · version control · CI · agent memory — nothing leaves your network.

# Summary

# Key Takeaways

**Lean code (ponytail)**

- Decision ladder before every change
- Runtime enforcement » documentation
- Remove dead code ruthlessly
- Type hints everywhere
- 46% fewer lines, 22% fewer tokens

**Multi-agent systems**

- gitea-mcp bridges agents to Gitea
- Same `AGENTS.md` for all tools
- `CONTEXT.md` per session
- Four memory types persist state
- Full self-hosted stack possible

**Links**: `github.com/DietrichGebert/ponytail` · `gitea.com/gitea/gitea-mcp` · `huggingface.co/Abzu`